DevOps Troubleshooting — Part 2

Prometheus, Grafana, Argo CD, SonarQube, Nexus/Artifactory, Nginx, Deep Networking, Security, Cost & Reliability

1. Prometheus is running, but one application target is DOWN. What will you check?
Answer:
First, I check the target status in Prometheus and identify the exact scrape error. Then I verify whether the application metrics endpoint is reachable from the Prometheus server and whether the configured host, port, and path are correct.
I also check service discovery, network connectivity, firewall rules, and whether the application is actually exposing /metrics.
After fixing the connectivity or configuration issue, I verify that the target becomes UP and metrics are being collected.
Simple ga gurthupettuko:
Prometheus target DOWN → target error chudu → /metrics reachable aa check cheyyi → network/config verify cheyyi → target UP ayyindo verify cheyyi.
Eng: First I check the scrape error, verify the metrics endpoint and connectivity, fix the issue, and confirm that the target becomes UP.
Example:
Prometheus → Status → Targets → DOWN target open chesi → Last Scrape and Error check cheyyi.
curl http://<app-ip>:<port>/metrics → metrics endpoint reachable ga undho check cheyyi.
curl -v http://<app-ip>:<port>/metrics → connection or HTTP error details chudu.
nc -vz <app-ip> <port> → Prometheus nunchi application port reachable aa check cheyyi.
journalctl -u prometheus -n 100 --no-pager → Prometheus-side errors chudu.
Issue endpoint/network ayithe → config or connectivity fix chesi → Prometheus Targets lo UP verify cheyyi.

---

2. Prometheus is showing context deadline exceeded **while scraping a target. What will you check?**
Answer:
First, I check whether the target is responding slowly or not responding at all. Then I verify network latency, application performance, firewall rules, and the scrape timeout configuration.
I also check whether the metrics endpoint itself is taking too long because of expensive metric collection.
After identifying the bottleneck, I fix the application or network issue and verify successful scrapes.
Simple ga gurthupettuko:
Timeout → target slow aa check → network/app metrics endpoint check → timeout reason fix → scrape verify.
Eng: I check why the metrics endpoint is taking too long and verify network and application performance.
Example:
curl -w "\n%{time_total}\n" http://<target>:<port>/metrics → metrics response time chudu.
ping <target> → basic latency check cheyyi.
nc -vz <target> <port> → port reachable aa check cheyyi.
Prometheus → Status → Targets → scrape error verify cheyyi.
Application logs → metrics endpoint processing slow ga undho chudu.
Issue expensive metrics ayithe → application/config optimize chesi → successful scrape verify cheyyi.

---

3. Prometheus has suddenly stopped collecting metrics from multiple applications. What will you investigate?
Answer:
First, I check whether the issue affects all targets or only a specific group. If multiple targets fail together, I investigate Prometheus itself, service discovery, network connectivity, configuration changes, and resource usage.
I also check whether Prometheus is running out of disk or memory or whether its configuration was recently changed.
After identifying the common failure, I fix it and confirm that multiple targets are scraping normally.
Simple ga gurthupettuko:
Multiple targets DOWN → common problem identify → Prometheus/network/config/resources check → fix → targets verify.
Eng: I look for a common cause affecting multiple targets instead of troubleshooting each application separately.
Example:
systemctl status prometheus → Prometheus running lo undho check cheyyi.
journalctl -u prometheus -n 100 --no-pager → service errors chudu.
df -h → disk full ayinda check cheyyi.
free -m → memory pressure unda check cheyyi.
curl http://localhost:9090/-/ready → Prometheus ready ga undho check cheyyi.
Prometheus → Status → Targets → multiple target errors compare cheyyi.
Common config/resource issue ayithe → fix chesi → target recovery verify cheyyi.

---

4. Prometheus disk usage is continuously increasing. What will you check?
Answer:
First, I check Prometheus data retention and determine how much historical data is being stored. Then I check whether retention settings are appropriate for the available disk capacity.
I also check whether the metric volume has increased because of high-cardinality labels or newly added targets.
I adjust retention or storage capacity and reduce unnecessary metrics if required.
Simple ga gurthupettuko:
Prometheus disk increase → retention check → metric volume/cardinality check → storage/retention optimize.
Eng: I check retention, metric volume, and high-cardinality data before changing storage.
Example:
df -h → Prometheus disk usage check cheyyi.
du -sh /var/lib/prometheus → data directory size chudu.
Prometheus configuration → --storage.tsdb.retention.time check cheyyi.

Prometheus UI → query count({__name__=~".+"}) → metric volume estimate cheyyi.
du -sh /var/lib/prometheus/* → storage consumption inspect cheyyi.
Unnecessary retention/metrics ayithe → retention reduce or metrics optimize chesi → disk usage monitor cheyyi.

---

5. Prometheus queries are becoming very slow. What will you investigate?
Answer:
First, I identify which queries are slow and whether the problem is limited to a specific dashboard or affects Prometheus generally.
Then I check query complexity, high-cardinality labels, time ranges, and Prometheus CPU and memory usage.
I optimize expensive queries and reduce unnecessary cardinality where possible.
Simple ga gurthupettuko:
Query slow → query identify → cardinality/time range/resources check → query optimize.
Eng: I identify expensive queries and check cardinality, query range, and Prometheus resources.
Example:
Prometheus UI → Graph → slow query run chesi execution behavior observe cheyyi.
top → Prometheus CPU/memory usage chudu.
free -m → memory pressure check cheyyi.
Query lo unnecessary large time range remove cheyyi.
High-cardinality labels identify chesi → unnecessary labels reduce cheyyi → query response verify cheyyi.

---

6. A Prometheus alert is not firing even though the metric is above the threshold. What will you check?
Answer:
First, I execute the alert expression manually and confirm that it returns the expected result. Then I check the alert rule configuration, evaluation interval, labels, and whether Prometheus has loaded the latest rule file.
I also verify Alertmanager connectivity if the alert is firing in Prometheus but notifications are not being delivered.
Simple ga gurthupettuko:
Alert fire avvakapothe → expression check → rule check → Prometheus reload check → Alertmanager check.
Eng: I validate the expression first, then check the rule and Alertmanager path.
Example:
Prometheus UI → Alerts → alert state check cheyyi.
Prometheus UI → expression manually run chesi result verify cheyyi.
promtool check rules <rules-file.yml> → rule syntax validate cheyyi.
curl http://localhost:9090/api/v1/rules → loaded rules verify cheyyi.
curl http://localhost:9090/api/v1/alerts → active alerts chudu.
Alert active but notification lekapothe → Alertmanager logs/config check cheyyi.

---

7. Grafana dashboard suddenly shows "No Data." What will you check?
Answer:
First, I check whether the Grafana data source is healthy. Then I verify the selected time range, dashboard variables, query, and whether Prometheus contains data for that period.
I also check whether the metric name or labels changed recently.
After fixing the query or data source issue, I refresh the dashboard and verify the panels.
Simple ga gurthupettuko:
Grafana No Data → data source check → time range/query check → Prometheus data verify → dashboard fix.
Eng: I check the data source, time range, query, and underlying Prometheus data.
Example:
Grafana → Connections → Data Sources → Prometheus → Save & Test click cheyyi.
Dashboard → Time Range → Last 1 hour select cheyyi.
Panel → Edit → Query inspect cheyyi.
Prometheus UI → same query run chesi data undho chudu.
Metric/label change ayithe → query update chesi → dashboard refresh cheyyi.

---

8. Grafana can connect to Prometheus, but dashboards are showing incorrect values. What will you check?
Answer:
First, I compare the Grafana query with the raw Prometheus query result. Then I check aggregation, labels, units, rate calculations, and dashboard variables.
I also verify whether the panel is displaying bytes, percentages, seconds, or another unit correctly.
After correcting the query or visualization, I compare the dashboard result with the source metric.
Simple ga gurthupettuko:
Wrong value → Grafana query compare → aggregation/unit/label check → Prometheus raw value verify → fix.
Eng: I compare the Grafana query with Prometheus directly and validate aggregation and units.
Example:
Grafana → Panel → Edit → Query copy cheyyi.
Prometheus UI → same query run chesi raw result compare cheyyi.
rate() / irate() / sum() usage verify cheyyi.
Grafana → Field → Unit → percentage/bytes/seconds correct ga set cheyyi.
Query correction tarvatha → panel refresh chesi → value verify cheyyi.

---

9. Grafana alert is working, but notifications are not reaching the team. What will you check?
Answer:
First, I verify whether the alert is actually firing in Grafana. Then I check the notification contact point, routing rules, notification policy, and destination configuration.

I also check Grafana logs for authentication, connectivity, or delivery errors.
After fixing the contact point or routing, I send a test notification and verify delivery.
Simple ga gurthupettuko:
Alert fire avutundi, notification ledu → contact point → routing → destination → logs → test notification.
Eng: I verify the alert state, routing, contact point, destination, and delivery logs.
Example:
Grafana → Alerting → Alert Rules → alert state check cheyyi.
Grafana → Alerting → Contact Points → destination verify cheyyi.
Grafana → Alerting → Notification Policies → routing check cheyyi.
journalctl -u grafana -n 100 --no-pager → Grafana notification errors chudu.
Contact Point → Test → notification delivery verify cheyyi.

---

10. Grafana is using very high memory. What will you investigate?
Answer:
First, I check whether the high memory usage is caused by dashboards, queries, alert evaluations, or a large number of concurrent users.
Then I identify expensive dashboards and queries and check Grafana logs.
I optimize heavy queries and dashboards and increase resources only if the workload genuinely requires it.
Simple ga gurthupettuko:
Grafana memory high → process/resource check → heavy dashboard/query identify → optimize → resources verify.
Eng: I identify the workload causing memory growth before increasing Grafana resources.
Example:
free -m → memory usage chudu.
ps aux --sort=-%mem | head → memory-consuming processes identify cheyyi.
Grafana dashboard → heavy panels identify cheyyi.
Grafana logs → journalctl -u grafana -n 100 --no-pager chudu.
Heavy query/panel ayithe → query optimize chesi → memory usage monitor cheyyi.

Argo CD / GitOps
11. Argo CD application shows OutOfSync**. What will you check?**
Answer:
First, I compare the desired state from Git with the live Kubernetes state. Then I identify the exact resource and field causing the difference.
I check whether the difference is caused by a Git change, manual Kubernetes change, generated fields, or an ignore-differences configuration.
Based on the cause, I update Git or safely sync the application.
Simple ga gurthupettuko:
OutOfSync → Git desired state vs cluster live state compare → difference identify → correct source → sync.
Eng: I identify exactly why the live state differs from Git before syncing.
Example:
argocd app get <app-name> → application status and sync state chudu.
argocd app diff <app-name> → Git and live state difference identify cheyyi.
kubectl get <resource> -o yaml → live resource inspect cheyyi.
Git manifest → desired configuration compare cheyyi.
Correct source decide chesi → argocd app sync <app-name> → Synced state verify cheyyi.

---

12. Argo CD application is stuck in Progressing**. What will you check?**
Answer:
First, I identify which resource is still progressing. Then I check the Pod, Deployment, ReplicaSet, events, probes, image, and resource availability.
I also verify whether the rollout is waiting because of readiness failures or insufficient cluster capacity.
After fixing the workload issue, I verify that Argo CD reports the application as Healthy.
Simple ga gurthupettuko:
Progressing → exact resource identify → Pod/events/probes/image/resources check → fix → Healthy verify.
Eng: I identify the resource preventing the application from becoming Healthy and troubleshoot it at Kubernetes level.
Example:
argocd app get <app-name> → resource status chudu.
kubectl get pods -n <namespace> → Pod status check cheyyi.
kubectl describe pod <pod> -n <namespace> → events/probes chudu.
kubectl logs <pod> -n <namespace> → application error check cheyyi.
kubectl rollout status deployment/<name> -n <namespace> → rollout status verify cheyyi.
Issue fix ayyaka → Argo CD application Healthy ga undho verify cheyyi.

---

13. Argo CD sync operation fails. What will you do?
Answer:
First, I inspect the sync error and identify which resource failed. Then I check Kubernetes API errors, RBAC permissions, manifests, dependencies, and resource conflicts.
I correct the underlying issue rather than repeatedly clicking Sync.
After fixing it, I run the sync again and verify the final application health.
Simple ga gurthupettuko:
Sync fail → error/resource identify → K8s error/RBAC/manifest check → fix → sync → Healthy.

Eng: I troubleshoot the failed Kubernetes resource first and then retry the Argo CD sync.
Example:
argocd app sync <app-name> → sync error capture cheyyi.
argocd app get <app-name> → failed resource identify cheyyi.
kubectl describe <resource> <name> -n <namespace> → resource error chudu.
kubectl auth can-i create <resource> -n <namespace> → permission check cheyyi.
Manifest fix chesi → sync rerun chesi → application health verify cheyyi.

---

14. Argo CD repository connection is failing. What will you check?
Answer:
First, I verify the repository URL and authentication configuration. Then I check network connectivity, DNS, credentials, certificates, and repository access from the Argo CD components.
I also check whether the repository is reachable and whether the configured credentials still have access.
After fixing the connection issue, I refresh the application and verify successful repository access.
Simple ga gurthupettuko:
Repo fail → URL → credentials → DNS/network → certificate/access → refresh.
Eng: I verify repository URL, authentication, connectivity, and certificates.
Example:
argocd repo list → repository connection status chudu.
argocd repo get <repo-url> → repository details verify cheyyi.
nslookup <git-host> → DNS check cheyyi.
curl -Iv https://<git-host> → HTTPS connectivity/certificate check cheyyi.
Argo CD repository credentials verify/update chesi → repo connection successful ga undho verify cheyyi.

---

15. Argo CD keeps reverting a manual Kubernetes change. Why?
Answer:
This is expected GitOps behavior when automated synchronization is enabled. Argo CD continuously compares the live cluster with the Git desired state and restores the Git version.
I identify whether the change should be permanent. If yes, I update Git instead of manually changing the cluster.
Simple ga gurthupettuko:
Manual change revert → Argo CD Git state follow chestundi → permanent change ayithe Git lo update cheyyi.
Eng: Argo CD restores the Git-defined state, so permanent changes should be made in Git.
Example:
argocd app get <app-name> → automated sync enabled aa check cheyyi.
argocd app diff <app-name> → Git vs live difference chudu.
Git manifest → required change add cheyyi.
Git commit/push → Argo CD refresh/sync → new desired state verify cheyyi.

SonarQube
16. SonarQube quality gate is failing even though the application builds successfully. What will you check?
Answer:
First, I check the SonarQube analysis results to identify the exact quality gate condition that failed.
Then I review bugs, vulnerabilities, code smells, coverage, duplication, and new-code conditions.
I fix the actual quality issue and rerun the analysis rather than bypassing the quality gate.
Simple ga gurthupettuko:
Build success but quality fail → Sonar result chudu → failed condition identify → code fix → scan rerun.
Eng: A successful build does not guarantee a successful quality gate, so I investigate the specific SonarQube condition.
Example:
SonarQube → Project → Quality Gate → failed condition identify cheyyi.
SonarQube → Issues → Bugs/Vulnerabilities/Code Smells chudu.
SonarQube → Measures → Coverage and duplication check cheyyi.
mvn sonar:sonar → Maven project analysis run cheyyi.
Issue fix chesi → scan rerun chesi → Quality Gate PASS verify cheyyi.

---

17. SonarQube scanner cannot connect to the SonarQube server. What will you check?
Answer:
First, I verify the SonarQube server URL configured for the scanner. Then I check DNS, network connectivity, authentication, TLS certificates, and server availability.
I also check scanner logs to identify whether the problem is connectivity or authentication related.
Simple ga gurthupettuko:
Scanner connect fail → URL → DNS/network → auth → TLS → server logs.
Eng: I verify the server URL, connectivity, authentication, and TLS before rerunning the scan.
Example:
curl -Iv https://<sonarqube-url> → server connectivity and TLS check cheyyi.
nslookup <sonarqube-host> → DNS verify cheyyi.
systemctl status sonarqube → server running aa check cheyyi.
Scanner output → exact authentication/connection error chudu.
Correct URL/token/certificate fix chesi → scanner rerun cheyyi.

---

18. SonarQube analysis is failing because the project has no coverage. What will you check?

Answer:
First, I verify whether tests are actually running and generating a coverage report. Then I check whether the scanner is pointing to the correct report path.
I also verify that the report format is supported by the configured analyzer.
After correcting the test or report configuration, I rerun the analysis.
Simple ga gurthupettuko:
Coverage ledu → tests run ayyaya → report generate ayyinda → path correct aa → scan rerun.
Eng: I verify test execution, coverage report generation, and the configured report path.
Example:
mvn test → tests successful ga run ayyaya check cheyyi.
find . -type f | grep -i coverage → coverage report search cheyyi.
find . -type f | grep -E 'jacoco.*xml' → JaCoCo report locate cheyyi.
Sonar scanner configuration → report path verify cheyyi.
Report fix chesi → mvn sonar:sonar → coverage verify cheyyi.

---

19. SonarQube scanner suddenly fails after a project upgrade. What will you compare?
Answer:
First, I compare the working and failing pipeline versions. Then I check the Java version, scanner version, build tool, SonarQube server compatibility, and project configuration.
I also review recent changes in the build files and scanner properties.
After identifying the incompatible component, I align the versions and rerun the analysis.
Simple ga gurthupettuko:
Upgrade tarvatha scanner fail → old/new versions compare → Java/scanner/server/build config check → compatible versions use cheyyi.
Eng: I compare the toolchain versions and project configuration between the working and failing builds.
Example:
java -version → Java version check cheyyi.
mvn -version → Maven and Java details chudu.
sonar-scanner --version → scanner version verify cheyyi.
SonarQube → Administration/System → server version check cheyyi.
Pipeline previous successful build tho versions compare chesi → compatible configuration fix cheyyi.

Nexus / JFrog Artifactory
20. Jenkins cannot download an artifact from Nexus. What will you check?
Answer:
First, I verify the repository URL and artifact coordinates. Then I check repository availability, authentication, permissions, network connectivity, and whether the requested version exists.
I also verify whether the artifact was deleted or moved.
After fixing the repository or credential issue, I retry the dependency download.
Simple ga gurthupettuko:
Artifact download fail → URL/artifact version → repo → credentials → permissions → network check.
Eng: I verify the artifact coordinates, repository availability, authentication, and permissions.
Example:
curl -I https://<nexus-url>/repository/<repo>/<artifact> → artifact URL response check cheyyi.
nslookup <nexus-host> → DNS check cheyyi.
curl -u <user>:<password> -I <artifact-url> → authentication test cheyyi.
Nexus → Browse → repository → artifact/version exists aa chudu.
Permission/URL issue fix chesi → Jenkins build rerun cheyyi.

---

21. Artifact upload to JFrog Artifactory is failing with 403 Forbidden**. What will you check?**
Answer:
A 403 usually indicates that the credentials are recognized but the user or token does not have permission for the requested operation.
I check the repository permissions, target path, user/token, and repository configuration.
After granting only the required permission, I retry the upload.
Simple ga gurthupettuko:
403 → authentication work ayye chance undi → permissions/path/repository check → required access fix → upload retry.
Eng: I focus on authorization, repository permissions, and target path when I receive a 403.
Example:
curl -u <user>:<token> -I https://<artifactory-url>/artifactory/<repo>/ → authentication test cheyyi.
Artifactory → Administration → Permissions → repository access check cheyyi.
Target repository/path correct aa verify cheyyi.
Jenkins credential/token permissions verify cheyyi.
Required permission fix chesi → artifact upload rerun cheyyi.

---

22. Artifact repository is running out of disk space. What will you investigate?
Answer:
First, I identify which repositories and artifacts are consuming most of the storage. Then I review retention policies, old versions, snapshots, build artifacts, and cleanup policies.
I safely remove only artifacts that are no longer required and configure retention to prevent repeated growth.
I also verify that repository functionality returns to normal.

Simple ga gurthupettuko:
Repo disk full → storage consumer identify → old artifacts/retention check → cleanup → monitor.
Eng: I identify storage consumers and apply controlled retention rather than deleting artifacts blindly.
Example:
df -h → repository server disk check cheyyi.
du -sh <repository-data-directory>/* → storage consumers chudu.
Nexus/Artifactory → repository storage usage check cheyyi.
Old snapshots/build artifacts identify cheyyi.
Retention/cleanup policy configure chesi → unnecessary artifacts clean cheyyi → df -h tho verify cheyyi.

Nginx / Load Balancing
23. Nginx is returning 502 Bad Gateway**. What will you check?**
Answer:
First, I check the Nginx error log to identify the upstream failure. Then I verify whether the backend application is running, reachable, listening on the expected port, and accepting connections.
I also check upstream configuration, DNS resolution, and network connectivity.
After fixing the backend or upstream configuration, I reload Nginx and verify the response.
Simple ga gurthupettuko:
502 → Nginx backend ki connect avvalekapothundi → upstream/app/port/network check → fix → test.
Eng: I check the Nginx error log and verify connectivity between Nginx and the upstream application.
Example:
tail -100 /var/log/nginx/error.log → exact 502 reason chudu.
ss -lntp → backend expected port lo listen chesthunda check cheyyi.
curl -v http://127.0.0.1:<backend-port> → backend response test cheyyi.
nc -vz <backend-ip> <port> → upstream connectivity check cheyyi.
nginx -t → Nginx configuration validate cheyyi.
Fix tarvatha → systemctl reload nginx → curl -I http://<host> → 502 resolve ayyindo verify cheyyi.

---

24. Nginx is returning 503 Service Unavailable**. What will you check?**
Answer:
First, I determine whether Nginx itself is unavailable or whether the upstream pool has no healthy backend servers.
Then I check upstream configuration, backend health, connection limits, and Nginx logs.
I restore at least one healthy backend or correct the upstream configuration and verify the response.
Simple ga gurthupettuko:
503 → backend healthy servers unnaya check → upstream/config/logs → healthy backend restore → verify.
Eng: I check whether Nginx has any healthy upstream servers available.
Example:
tail -100 /var/log/nginx/error.log → upstream error chudu.
nginx -T | grep -A10 upstream → upstream configuration inspect cheyyi.
curl http://<backend-ip>:<port>/health → backend health check cheyyi.
ss -s → connection state check cheyyi.
Backend/config fix chesi → nginx -t && systemctl reload nginx → response verify cheyyi.

---

25. Nginx is returning 504 Gateway Timeout**. What will you investigate?**
Answer:
A 504 usually indicates that Nginx did not receive a response from the upstream within the configured timeout.
I check backend response time, application logs, database calls, network latency, and Nginx timeout settings.
I fix the underlying slow dependency first and change timeout values only when appropriate.
Simple ga gurthupettuko:
504 → backend response late → app/db/network slow aa check → root cause fix → timeout only if required.
Eng: I identify why the upstream is slow before simply increasing the timeout.
Example:
tail -100 /var/log/nginx/error.log → timeout details chudu.
time curl -v http://<backend>:<port>/api → backend response time measure cheyyi.
curl -v http://<backend>:<port>/health → backend basic response test cheyyi.
Application logs → slow DB/API calls check cheyyi.
nginx -T | grep -i timeout → timeout configuration inspect cheyyi.
Root cause fix chesi → request retest cheyyi.

---

26. Nginx SSL certificate has expired. What will you do?
Answer:
First, I confirm the certificate expiry date and identify which certificate Nginx is serving.
Then I install the renewed certificate and private key securely, validate the Nginx configuration, and reload Nginx.
Finally, I verify the certificate from the client side.
Simple ga gurthupettuko:
SSL expired → certificate expiry confirm → renewed cert install → config test → reload → verify.
Eng: I renew the certificate, validate the configuration, reload Nginx, and verify the served certificate.
Example:
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null | openssl x509 -noout -dates → expiry dates chudu.
nginx -T | grep -E 'ssl_certificate|ssl_certificate_key' → certificate paths identify cheyyi.

Renewed certificate correct path lo place cheyyi.
nginx -t → configuration validate cheyyi.
systemctl reload nginx → certificate reload cheyyi.
openssl s_client ... → new expiry date verify cheyyi.

---

27. Load balancer is sending traffic to an unhealthy backend. What will you check?
Answer:
First, I check the load balancer health-check configuration, including protocol, port, path, expected status code, and timeout.
Then I test the same health endpoint directly on the backend and verify security rules and application readiness.
I correct the health check or backend issue and confirm that unhealthy instances are removed from traffic.
Simple ga gurthupettuko:
LB wrong traffic → health check config → backend endpoint → port/security → fix → healthy status verify.
Eng: I compare the load balancer health check with the backend's actual health endpoint.
Example:
curl -v http://<backend-ip>:<port>/health → backend health endpoint test cheyyi.
LB console → target health → failed reason chudu.
Health check port/path/protocol compare cheyyi.
ss -lntp → backend port listening aa check cheyyi.
Security rules verify chesi → fix → target becomes healthy ani verify cheyyi.

Deep Networking
28. Application hostname is not resolving, but the server IP works. What will you check?
Answer:
First, I verify DNS resolution from the affected client and compare it with a working machine.
Then I check the DNS record, resolver configuration, TTL, search domain, and whether the hostname resolves to the correct IP.
After correcting the DNS record or client configuration, I verify resolution again.
Simple ga gurthupettuko:
IP work, hostname fail → DNS check → record/resolver/TTL check → fix → resolve verify.
Eng: I determine whether the problem is with the DNS record or the client's DNS resolver.
Example:
nslookup app.example.com → DNS resolution check cheyyi.
dig app.example.com → DNS response detailed ga chudu.
cat /etc/resolv.conf → configured DNS servers check cheyyi.
dig @<dns-server> app.example.com → specific resolver test cheyyi.
Correct DNS record/resolver fix chesi → dig app.example.com → expected IP verify cheyyi.

---

29. DNS resolves correctly, but the application is still unreachable. What will you check?
Answer:
Since DNS is working, I move to the network and application layers.
I check whether the destination port is open, whether the service is listening, firewall/security rules, routing, and application health.
I test connectivity from the actual client or source environment.
Simple ga gurthupettuko:
DNS okay → port check → service listening → firewall/routing → application check.
Eng: After DNS is confirmed, I troubleshoot TCP connectivity and the application itself.
Example:
dig app.example.com → IP confirm cheyyi.
nc -vz app.example.com 443 → TCP port connectivity check cheyyi.
curl -vk https://app.example.com → HTTP/TLS response check cheyyi.
ss -lntp → server listening port verify cheyyi.
ip route → routing table check cheyyi.
Firewall/security rule issue ayithe → required access fix chesi → curl retest cheyyi.

---

30. TCP connection is timing out. What will you check?
Answer:
First, I determine whether the client can reach the destination IP and whether the destination port is listening.
Then I check routing, firewall/security rules, network ACLs, and intermediate network controls.
I compare the behavior from another source to identify whether the issue is source-specific or destination-wide.
Simple ga gurthupettuko:
TCP timeout → IP reachability → port → route → firewall/security → compare source.
Eng: I trace the connection from the client to the destination and identify where packets are being blocked.
Example:
ping <destination-ip> → basic reachability test cheyyi.
nc -vz <destination-ip> <port> → TCP connection test cheyyi.
ip route get <destination-ip> → selected route chudu.
traceroute <destination-ip> → network path inspect cheyyi.
Server side → ss -lntp → port listening aa check cheyyi.
Firewall/security rules inspect chesi → blocked path fix → nc retest cheyyi.

31. Application is reachable locally but not from another server. What will you check?
Answer:
First, I confirm that the application is listening on the correct interface and not only on localhost.
Then I check the server firewall, network security rules, routing, and destination port.
I test from both localhost and the remote server to isolate the network boundary causing the problem.
Simple ga gurthupettuko:
Local work, remote fail → bind address → firewall → route → port → remote test.
Eng: I check whether the service is exposed beyond localhost and whether network controls allow access.
Example:
ss -lntp | grep <port> → service 127.0.0.1 ki bind ayinda or 0.0.0.0 ki bind ayinda chudu.
curl http://127.0.0.1:<port> → local test cheyyi.
Remote server → nc -vz <server-ip> <port> → remote connectivity test cheyyi.
ip route → route check cheyyi.
Firewall rules verify chesi → correct interface/firewall config fix cheyyi → remote curl verify cheyyi.

---

32. HTTP request returns 401 Unauthorized**. What will you check?**
Answer:
A 401 generally indicates that authentication is missing or invalid.
I check the Authorization header, token validity, credentials, token expiry, authentication configuration, and whether the request is reaching the expected service.
I test with valid credentials and verify the authentication flow.
Simple ga gurthupettuko:
401 → authentication problem → token/header/credentials/expiry check → valid auth tho test.
Eng: I verify the authentication credentials and token before investigating application authorization.
Example:
curl -v https://<host>/api → response headers and 401 chudu.
curl -H "Authorization: Bearer <token>" https://<host>/api → token-based request test cheyyi.
Token expiry/validity check cheyyi.
Application/auth logs → authentication failure reason chudu.
Correct credential/token use chesi → API response verify cheyyi.

---

33. HTTP request returns 403 Forbidden**. What will you check?**
Answer:
A 403 usually means the request reached the server but access was denied.
I check user permissions, IAM/RBAC or application authorization, IP restrictions, reverse-proxy rules, WAF policies, and file permissions where applicable.
I distinguish authentication failure from authorization failure before making changes.
Simple ga gurthupettuko:
403 → request reach ayindi but access denied → permissions/policy/IP/WAF check.
Eng: I investigate authorization and access-control rules rather than treating 403 as a connectivity issue.
Example:
curl -v https://<host>/path → HTTP response details chudu.
Nginx logs → tail -100 /var/log/nginx/access.log → request status check cheyyi.
tail -100 /var/log/nginx/error.log → access denial reason chudu.
Application authorization logs check cheyyi.
Required permission/policy fix chesi → request retest cheyyi.

Security
34. An application suddenly cannot access an AWS resource after an IAM policy change. What will you check?
Answer:
First, I identify the exact API operation that is failing. Then I check the IAM identity, attached policies, resource policy, conditions, and recent policy changes.
I also verify whether the application is using the expected role or credentials.
I restore only the minimum required permission and test the operation again.
Simple ga gurthupettuko:
IAM change → exact API failure → identity/policy/resource policy/conditions → least permission fix → test.
Eng: I identify the denied action and correct the minimum required IAM permission.
Example:
AWS CloudTrail → failed API event → exact denied action identify cheyyi.
aws sts get-caller-identity → application identity/role verify cheyyi.
aws iam list-attached-role-policies --role-name <role> → attached policies check cheyyi.
Policy action/resource/condition verify cheyyi.
Required permission matrame add chesi → API operation retest cheyyi.

---

35. Kubernetes application is getting Forbidden **errors after an RBAC change. What will you check?**
Answer:
First, I identify the exact Kubernetes API operation and resource being denied.
Then I check the ServiceAccount, Role or ClusterRole, and RoleBinding or ClusterRoleBinding.
I grant only the required permission and verify access using Kubernetes authorization checks.
Simple ga gurthupettuko:
K8s Forbidden → exact resource/action → ServiceAccount → Role/Binding → minimum permission → test.

Eng: I identify the denied API action and correct the relevant RBAC binding.
Example:
kubectl auth can-i get pods --as=system:serviceaccount:<ns>:<sa> -n <ns> → permission test cheyyi.
kubectl get role,rolebinding -n <ns> → namespace RBAC check cheyyi.
kubectl get clusterrole,clusterrolebinding → cluster-level permissions check cheyyi.
Required rule/binding fix chesi → kubectl auth can-i ... → yes verify cheyyi.

---

36. TLS handshake is failing between two services. What will you check?
Answer:
First, I check whether the client and server can establish a TCP connection. Then I inspect certificate validity, hostname/SAN, trust chain, TLS versions, and cipher compatibility.
I also check whether an intermediate certificate is missing or whether a proxy is terminating TLS incorrectly.
After correcting the certificate or TLS configuration, I retest the handshake.
Simple ga gurthupettuko:
TLS fail → TCP first → certificate/SAN/chain → TLS version/cipher → fix → handshake verify.
Eng: I separate TCP connectivity problems from certificate and TLS negotiation problems.
Example:
nc -vz <host> 443 → TCP connectivity check cheyyi.
openssl s_client -connect <host>:443 -servername <host> → TLS handshake/certificate inspect cheyyi.
openssl x509 -in cert.pem -noout -dates -subject -issuer → certificate details chudu.
Certificate SAN/hostname verify cheyyi.
Trust chain/TLS configuration fix chesi → openssl s_client tho successful handshake verify cheyyi.

---

37. A secret was rotated, but the application still uses the old secret. What will you check?
Answer:
First, I identify how the application consumes the secret—environment variable, mounted file, external secret provider, or another mechanism. Then I verify that the new secret exists and that the application has access to it.
I also check whether the application or Pod needs to be restarted to load the new value. If the secret is injected through a mounted file, I verify whether the application rereads the file or requires a restart.
After updating the secret correctly, I restart or reload the application when required and verify that it is using the new credential.
Simple ga gurthupettuko:
Secret rotate chesina old value use chesthe → secret ekkada nunchi vastundo check → new value update ayinda check → app restart/reload required aa chudu → new secret work avutundho verify cheyyali.
Eng: I check how the application receives the secret, verify the new value, restart or reload the application if required, and confirm that the new credential is being used.
Example:
kubectl get secret <secret-name> -n <namespace> → Kubernetes Secret exists aa check cheyyi.
kubectl describe pod <pod-name> -n <namespace> → Secret or environment injection ela jarugutundo chudu.
kubectl exec <pod-name> -n <namespace> -- env → environment variable inject ayyinda check cheyyi.
kubectl rollout restart deployment/<deployment-name> -n <namespace> → application restart cheyyi if required.
kubectl rollout status deployment/<deployment-name> -n <namespace> → new Pods successfully ready ayyayo verify cheyyi.
Application logs → authentication successful ga undho verify cheyyi.

---

38. AWS EC2 instance is running, but the application is not reachable from the internet. What will you check?
Answer:
First, I verify whether the application is actually running and listening on the expected port on the EC2 instance. Then I check the instance's public IP or DNS, Security Group rules, subnet route table, and whether the subnet has a route to an Internet Gateway.
I also verify the operating-system firewall and whether the application is bound to the correct network interface instead of only localhost.
After identifying the blocked layer, I fix the required configuration and test connectivity from outside the instance.
Simple ga gurthupettuko:
EC2 running but app unreachable → app/port check → public IP → Security Group → route/Internet Gateway → OS firewall → bind address check.
Eng: I troubleshoot the application, port, Security Group, routing, public connectivity, and host firewall in that order.
Example:
aws ec2 describe-instances --instance-ids <instance-id> → instance state and public IP details chudu.
ss -lntp → application expected port lo listening undho check cheyyi.
curl http://localhost:<port> → application locally working aa check cheyyi.
curl http://<public-ip>:<port> → external connectivity test cheyyi.
AWS Console → EC2 → Security Groups → Inbound Rules → required port allowed aa check cheyyi.
ip route → instance routing table chudu.
sudo ufw status → OS firewall check cheyyi.
Issue fix chesi → external curl tho application reachable ayyindo verify cheyyi.

---

39. An AWS EC2 instance is running out of memory and applications are getting killed. What will you investigate?
Answer:
First, I check memory and swap usage and identify which processes are consuming the most memory. Then I check application logs for out-of-memory errors and determine whether memory usage is continuously increasing or caused by a temporary workload spike.
I also check whether the instance size is appropriate for the workload and whether application limits or memory leaks are present.

After identifying the cause, I optimize the application or workload and resize the instance only when additional capacity is genuinely required.
Simple ga gurthupettuko:
EC2 memory full → memory usage check → highest process identify → OOM logs check → workload/app issue fix → resize only if required.
Eng: I identify the process consuming memory, determine whether it is a leak or workload issue, and then optimize or scale appropriately.
Example:
free -m → total, used, available memory chudu.
top → memory-consuming process identify cheyyi.
ps aux --sort=-%mem | head → top memory processes chudu.
dmesg | grep -i "out of memory" → OOM events check cheyyi.
journalctl -k | grep -i "oom" → kernel OOM messages chudu.
Application logs → memory-related errors check cheyyi.
Issue application memory leak ayithe → application fix/restart cheyyi; capacity saripokapothe → suitable EC2 instance size ki scale cheyyi.

---

40. AWS ALB is returning 504 Gateway Timeout. What will you check?
Answer:
First, I verify whether the target instances or Pods are healthy. Then I check the target application's response time, target port, Security Groups, network connectivity, and application logs.
I also compare the application's response time with the ALB idle timeout. If the backend is taking too long because of database or external API calls, I fix the backend bottleneck rather than simply increasing the timeout.
Simple ga gurthupettuko:
ALB 504 → target health → backend response time → SG/network → app/DB dependency → timeout check.
Eng: I determine why the backend is not responding within the expected time and fix the backend or network issue.
Example:
AWS Console → EC2 → Target Groups → Targets → target health check cheyyi.
curl -v http://<target-private-ip>:<port>/health → backend health test cheyyi.
time curl http://<target-private-ip>:<port>/api → response time measure cheyyi.
AWS Console → ALB → Attributes → idle timeout verify cheyyi.
Application logs → slow request/database/API calls identify cheyyi.
Root cause fix chesi → ALB endpoint retest cheyyi.

---

41. AWS S3 upload suddenly fails with AccessDenied. What will you check?
Answer:
First, I identify which IAM identity is performing the S3 operation and the exact API action that is denied. Then I check IAM policies, the S3 bucket policy, Block Public Access settings where relevant, object ownership, and any explicit deny conditions.
I also verify that the application is using the expected IAM role and that the requested bucket and key path are correct.
I grant only the required permission and test the upload again.
Simple ga gurthupettuko:
S3 AccessDenied → caller identity → exact action → IAM policy → bucket policy → explicit deny → minimum permission fix.
Eng: I identify the denied S3 action and check both identity-based and bucket-based permissions before changing access.
Example:
aws sts get-caller-identity → current IAM identity/role verify cheyyi.
aws s3 ls s3://<bucket-name> → bucket access test cheyyi.
aws s3 cp test.txt s3://<bucket-name>/<path>/ → upload test cheyyi.
AWS CloudTrail → failed S3 event → exact denied action chudu.
AWS Console → S3 → Permissions → Bucket Policy inspect cheyyi.
IAM policy lo required action/resource correct ga undho verify chesi → minimum permission fix cheyyi → upload retest cheyyi.

---

42. AWS VPC application cannot connect to a database in another subnet. What will you check?
Answer:
First, I verify that both subnets have the correct route tables and that the database is reachable on its expected port. Then I check Security Groups, Network ACLs, DNS resolution, and whether the database is listening.
I also verify that the application is using the correct database endpoint and port.
After identifying the network or security restriction, I correct it and test connectivity from the application subnet.
Simple ga gurthupettuko:
App → DB connect fail → endpoint/port → route table → Security Group → NACL → DNS → DB listening check.
Eng: I verify routing, security controls, DNS, and the database listener from the application's network location.
Example:
nslookup <db-endpoint> → DB DNS resolve avutundha check cheyyi.
nc -vz <db-endpoint> 5432 → PostgreSQL connectivity test cheyyi.
nc -vz <db-endpoint> 3306 → MySQL connectivity test cheyyi.
ip route → application server route check cheyyi.
AWS Console → VPC → Route Tables → subnet routes verify cheyyi.
AWS Console → Security Groups → application and database SG rules compare cheyyi.
VPC → Network ACLs → inbound/outbound rules check cheyyi.
Fix tarvatha → nc/curl or application connection test rerun cheyyi.

43. AWS CloudWatch is not showing expected application metrics. What will you check?
Answer:
First, I identify whether the missing metric is a standard AWS metric, a custom metric, or an application-level metric. Then I check the metric namespace, dimensions, time range, and whether the monitoring agent or application is publishing the metric.
I also verify IAM permissions and CloudWatch Agent status if custom metrics are involved.
After correcting the publishing or configuration issue, I verify that new datapoints appear in CloudWatch.
Simple ga gurthupettuko:
CloudWatch metric missing → metric type → namespace/dimensions → agent → IAM → publishing check → new datapoint verify.
Eng: I verify whether the metric is being published correctly and check the namespace, dimensions, agent, and permissions.
Example:
AWS Console → CloudWatch → Metrics → correct namespace select cheyyi.
AWS Console → CloudWatch → Metrics → Dimensions verify cheyyi.
systemctl status amazon-cloudwatch-agent → agent running aa check cheyyi.
journalctl -u amazon-cloudwatch-agent -n 100 --no-pager → agent errors chudu.
aws cloudwatch list-metrics --namespace <namespace> → metric exists aa check cheyyi.
IAM role permissions verify chesi → agent/application restart or config fix → CloudWatch lo new datapoint verify cheyyi.

---

44. Kubernetes Pods are restarting because of OOMKilled. What will you check?
Answer:
First, I confirm that the Pod termination reason is actually OOMKilled. Then I check the container's memory usage, memory limits, application behavior, and whether the workload has increased.
I compare the actual memory requirement with the configured requests and limits. If the application genuinely requires more memory, I adjust the resources appropriately; if memory usage is abnormal, I investigate the application.
Simple ga gurthupettuko:
OOMKilled → memory usage → limits → workload → application leak/high usage → resources or app fix.
Eng: I confirm the OOM condition, compare actual usage with container limits, and determine whether the problem is application behavior or insufficient resources.
Example:
kubectl get pod <pod> -n <namespace> → Pod status check cheyyi.
kubectl describe pod <pod> -n <namespace> → Last State and OOMKilled reason chudu.
kubectl top pod <pod> -n <namespace> → current memory usage chudu.
kubectl get pod <pod> -n <namespace> -o yaml → memory requests/limits inspect cheyyi.
Application logs → memory-related errors check cheyyi.
Limit too low ayithe → resource limit adjust chesi → rollout verify cheyyi.

---

45. Kubernetes Service exists, but Pods are not receiving traffic. What will you check?
Answer:
First, I check whether the Service has any endpoints. Then I compare the Service selector with the labels on the Pods and verify that the Pods are Ready.
I also check the Service port and targetPort and test connectivity from inside the cluster.
After correcting the selector, readiness, or port configuration, I verify that endpoints are populated and traffic reaches the Pods.
Simple ga gurthupettuko:
Service exists but traffic ledu → endpoints check → selector/labels compare → Pod Ready check → port/targetPort check → test.
Eng: I first verify that the Service has healthy endpoints and that its selector and ports match the Pods.
Example:
kubectl get svc <service> -n <namespace> → Service ports chudu.
kubectl get endpoints <service> -n <namespace> → endpoints unnaya check cheyyi.
kubectl get pods -n <namespace> --show-labels → Pod labels chudu.
kubectl describe svc <service> -n <namespace> → selector and targetPort verify cheyyi.
kubectl get pods -n <namespace> → Ready status check cheyyi.
kubectl run tmp --rm -it --image=curlimages/curl -- sh → cluster lopala test Pod create cheyyi.
curl http://<service>:<port> → Service traffic test cheyyi.

---

46. Kubernetes Ingress is returning 404 even though the application Service works. What will you check?
Answer:
Since the Service works, I focus on the Ingress layer. I check the hostname, path rules, Ingress class, backend Service name, Service port, and whether the request Host header matches the configured rule.
I also check the Ingress Controller logs and confirm that the Ingress resource was accepted and configured correctly.
After correcting the rule or backend configuration, I test using the correct hostname and path.
Simple ga gurthupettuko:
Service work, Ingress 404 → host/path → Ingress class → backend service/port → controller logs → correct Host tho test.
Eng: I troubleshoot the Ingress rules and controller configuration because the backend Service is already working.
Example:
kubectl get ingress -n <namespace> → host/path/backend chudu.
kubectl describe ingress <ingress-name> -n <namespace> → events and backend details inspect cheyyi.
kubectl get ingressclass → IngressClass verify cheyyi.
kubectl get svc -n <namespace> → backend Service/port verify cheyyi.
kubectl logs -n <ingress-namespace> <ingress-controller-pod> → controller errors chudu.
curl -H "Host: app.example.com" http://<ingress-ip>/<path> → correct host/path tho test cheyyi.

47. AWS cost suddenly increased significantly. How will you investigate?
Answer:
First, I identify which AWS service, account, region, or resource caused the increase. Then I compare current usage with the previous period and check for unexpected EC2 instances, large data transfer, NAT Gateway usage, EBS volumes, snapshots, S3 storage, or other high-cost resources.
I also check whether a deployment or scaling event caused the increase.
After identifying the source, I remove unnecessary resources or optimize the workload without impacting Production availability.
Simple ga gurthupettuko:
AWS cost sudden increase → service identify → region/resource identify → usage compare → deployment/scaling/data transfer check → optimize.
Eng: I use cost and usage data to identify the exact service or resource responsible before taking corrective action.
Example:
AWS Console → Billing → Cost Explorer → current vs previous period compare cheyyi.
Cost Explorer → Group by Service → expensive service identify cheyyi.
Cost Explorer → Group by Region → high-cost region chudu.
aws ec2 describe-instances → unexpected instances check cheyyi.
AWS Console → EC2 → Volumes → unused EBS volumes identify cheyyi.
AWS Console → VPC → NAT Gateways → data processing usage check cheyyi.
Root cause identify chesi → unnecessary resource stop/delete or workload optimize → next usage monitor cheyyi.

---

48. Production application suddenly becomes very slow, but CPU and memory look normal. What will you investigate?
Answer:
If CPU and memory are normal, I move beyond host resources and investigate latency between dependencies. I check database response time, external API calls, network latency, DNS resolution, connection pools, disk I/O, and application thread or connection saturation.
I compare the current latency with a previously healthy period and identify which dependency is adding the delay.
After finding the bottleneck, I fix that dependency or configuration and verify application response time.
Simple ga gurthupettuko:
CPU/memory normal but app slow → dependency latency → DB/API/network/DNS/connection pool/I/O → bottleneck identify → fix → response verify.
Eng: I move from basic resource checks to dependency and latency analysis when CPU and memory are normal.
Example:
time curl https://<application>/health → application response time chudu.
curl -w "\nDNS:%{time_namelookup} Connect:%{time_connect} Total:%{time_total}\n" https://<application> → request timing breakdown chudu.
dig <dependency-host> → DNS response check cheyyi.
nc -vz <db-host> <db-port> → dependency connectivity check cheyyi.
Application logs → slow DB/API requests identify cheyyi.
Database monitoring → query latency check cheyyi.
Bottleneck fix chesi → response time compare cheyyi.

---

49. Production availability is dropping intermittently, but no single server is completely down. What will you investigate?
Answer:
First, I look for patterns in the failures instead of checking only whether individual servers are running. I correlate load balancer health checks, application logs, error rates, response latency, deployment events, and infrastructure metrics.
I check whether failures are isolated to a specific instance, Availability Zone, backend, dependency, or time period.
Once I identify the common pattern, I remove the unhealthy component or fix the underlying dependency and monitor availability after the change.
Simple ga gurthupettuko:
Intermittent outage → pattern identify → LB health → individual backend → logs/metrics → AZ/dependency/deployment compare → unhealthy component isolate.
Eng: I correlate logs, metrics, load-balancer health, and deployment events to identify the intermittent failure pattern.
Example:
AWS Console → ALB → Target Groups → target health history chudu.
kubectl get pods -A -o wide → affected Pods/nodes identify cheyyi.
kubectl get events -A --sort-by=.lastTimestamp → recent cluster events chudu.
curl -I https://<application> → repeated response test cheyyi.
Application logs → timestamps compare chesi errors identify cheyyi.
Prometheus → error rate/latency query chesi failure period compare cheyyi.
Affected backend/AZ identify ayithe → traffic drain/isolate chesi → availability recover ayyindo verify cheyyi.

---

50. A Production incident involves AWS, Kubernetes, and the application at the same time. How will you troubleshoot it?
Answer:
I follow a structured troubleshooting approach instead of changing multiple things at once. First, I understand the user impact and identify when the incident started. Then I check the application symptoms, Kubernetes health, AWS infrastructure, networking, and recent deployments or configuration changes.
I use logs, metrics, events, and health checks to narrow the problem to a specific layer. Once I identify the likely root cause, I take the safest recovery action, verify service health, and then perform a deeper root-cause analysis.
I also document the incident, timeline, root cause, recovery steps, and preventive actions.
Simple ga gurthupettuko:
Production incident → impact/time identify → recent changes check → app → K8s → AWS/network → logs + metrics + events → root cause → safest recovery → verify → RCA.
Eng: I troubleshoot layer by layer, correlate evidence across AWS, Kubernetes, and the application, recover safely, and document the root cause and preventive actions.
Example:
date → incident time and current system time verify cheyyi.
kubectl get pods -A → Kubernetes workload health check cheyyi.
kubectl get events -A --sort-by=.lastTimestamp → recent cluster events chudu.
kubectl logs <pod> -n <namespace> --since=30m → recent application errors chudu.
kubectl top nodes → node resource usage check cheyyi.
AWS Console → EC2/ALB/Target Groups → infrastructure and target health verify cheyyi.
CloudWatch → CPU, latency, error rate, network metrics compare cheyyi.
Prometheus/Grafana → incident time lo application/Kubernetes metrics correlate cheyyi.
git log -10 --oneline → recent deployment/code changes identify cheyyi.
Argo CD → application sync/deployment history check cheyyi.
Root cause identify chesi → safest recovery/rollback/fix perform chesi → application health, metrics, logs verify cheyyi → final ga RCA and preventive action document cheyyi.
Part-2 status
#1–50 complete. Ee Part-2 lo specifically **Prometheus, Grafana, Argo CD, SonarQube, Nexus/Artifactory, Nginx, deep networking, security, AWS, cost/reliability, and cross-layer Production incidents** cover ayyayi. Earlier Part-1 questions ni intentional ga repeat cheyyaledu.